Computer Vision

Exam Preparation Notes

B.Tech Semester Exam Quick-Reference — 12 Core Topics

Corrected Edition — with Python implementations of every algorithm

Table of Contents

1. Fundamentals of Image Formation

- 1.1 Introduction
- 1.2 Core Concepts
- 1.3 Mathematical Formulation
- 1.4 Matrix Representation & Worked Numerical Example
- 1.5 Formula Sheet
- 1.6 Applications
- 1.7 Advantages & Limitations
- 1.8 Common Mistakes
- 1.9 Key Points / Revision

2. Convolution and Filtering

- 2.1 Introduction
- 2.2 Core Concepts
- 2.3 Common Filters
- 2.4 Solved Numerical Example
- 2.5 Algorithm: 2D Convolution
- 2.6 Formula Sheet
- 2.7 Applications
- 2.8 Advantages & Limitations
- 2.9 Common Mistakes
- 2.10 Key Points / Revision

3. Image Enhancement

- 3.1 Introduction
- 3.2 Core Concepts
- 3.3 Solved Numerical Example — Contrast Stretching
- 3.4 Formula Sheet
- 3.5 Applications
- 3.6 Advantages & Limitations
- 3.7 Common Mistakes
- 3.8 Key Points / Revision

4. Depth Estimation

- 4.1 Introduction
- 4.2 Core Concepts
- 4.3 Mathematical Formulation
- 4.4 Solved Numerical Example
- 4.5 Algorithm: Stereo Depth Estimation
- 4.6 Formula Sheet
- 4.7 Applications

4.8 Advantages & Limitations
4.9 Common Mistakes
4.10 Key Points / Revision

5. Fundamental Matrix

5.1 Introduction
5.2 Core Concepts
5.3 Mathematical Formulation
5.4 Matrix Representation
5.5 Solved Numerical Example
5.6 Algorithm: Eight-Point Algorithm
5.7 Formula Sheet
5.8 Fundamental Matrix vs Essential Matrix
5.9 Applications
5.10 Advantages & Limitations
5.11 Common Mistakes
5.12 Key Points / Revision

6. Epipolar Geometry

6.1 Introduction
6.2 Core Concepts
6.3 Point Correspondence via Epipolar Geometry
6.4 Formula Sheet
6.5 Applications
6.6 Advantages & Limitations
6.7 Common Mistakes
6.8 Key Points / Revision

7. Homography

7.1 Introduction
7.2 Core Concepts
7.3 Mathematical Formulation
7.4 Matrix Representation & Worked Numerical Example
7.5 Algorithm: Estimating Homography
7.6 Formula Sheet
7.7 Applications
7.8 Advantages & Limitations
7.9 Common Mistakes
7.10 Key Points / Revision

8. Rectification

8.1 Introduction
8.2 Core Concepts
8.3 Mathematical Formulation

8.4 Image Alignment Pipeline
8.5 Algorithm: Stereo Rectification (calibrated case)
8.6 Formula Sheet
8.7 Applications
8.8 Advantages & Limitations
8.9 Common Mistakes
8.10 Key Points / Revision

9. Direct Linear Transformation (DLT)

9.1 Introduction
9.2 Core Concepts
9.3 Mathematical Formulation (Homography case)
9.4 Matrix Representation & Worked Numerical Example
9.5 Algorithm: DLT
9.6 Formula Sheet
9.7 Applications
9.8 Advantages & Limitations
9.9 Common Mistakes
9.10 Key Points / Revision

10. RANSAC (Random Sample Consensus)

10.1 Introduction
10.2 Core Concepts
10.3 Mathematical Formulation
10.4 Solved Numerical Example
10.5 Algorithm: RANSAC
10.6 Formula Sheet
10.7 Applications
10.8 Advantages & Limitations
10.9 Common Mistakes
10.10 Key Points / Revision

11. Camera Calibration

11.1 Introduction
11.2 Core Concepts
11.3 Mathematical Formulation
11.4 Matrix Representation
11.5 Solved Numerical Example
11.6 Algorithm: Zhang's Calibration
11.7 Formula Sheet
11.8 Applications
11.9 Advantages & Limitations
11.10 Common Mistakes

11.11 Key Points / Revision

12. Histogram Processing

12.1 Introduction

12.2 Core Concepts

12.3 Histogram Equalization

12.4 Histogram Equalization vs Histogram Matching

12.5 Solved Numerical Example — Histogram Equalization

12.6 Formula Sheet

12.7 Algorithm: Histogram Equalization

12.8 Applications

12.9 Advantages & Limitations

12.10 Common Mistakes

12.11 Key Points / Revision

1. Fundamentals of Image Formation

1.1 Introduction

Definition: Image formation is the process by which a 3D scene in the real world is projected onto a 2D image plane through a camera (or an equivalent optical system).

Why it matters: Every computer vision algorithm — from edge detection to 3D reconstruction — assumes a mathematical model of how points in space become pixels in an image. Without this model, we cannot relate image measurements back to real-world geometry.

Applications: camera design, robot vision, AR/VR head tracking, autonomous vehicle perception, photogrammetry, 3D reconstruction.

1.2 Core Concepts

1.2.1 Pinhole Camera Model

The simplest camera model. Light from a 3D point passes through a single infinitesimal aperture (the pinhole / optical center) and forms an inverted image on the image plane located at a distance f (focal length) behind the pinhole. In practice we use the virtual image plane placed in front of the pinhole (at distance f) so the image is upright — this is mathematically equivalent and is the standard convention used in vision.

1.2.2 Coordinate Systems

Four coordinate systems are involved, in order:

- World coordinates (Xw, Yw, Zw) — fixed reference frame of the scene.
- Camera coordinates (Xc, Yc, Zc) — origin at camera optical center, Zc axis = optical axis.
- Image coordinates (x, y) — 2D coordinates on the image plane (physical units, e.g. mm), origin at principal point.
- Pixel coordinates (u, v) — discrete pixel grid, origin usually top-left corner.

Pipeline: World Coordinates --(R,t)--> Camera Coordinates --(Persp. Proj.)--> Image (Retinal) Coordinates --(K, intrinsics)--> Pixel Coordinates

1.2.3 Perspective Projection

A 3D point projects onto the image plane by dividing by depth (Zc) — this is what makes far objects appear smaller (the perspective effect).

1.2.4 Homogeneous Coordinates

Ordinary Euclidean coordinates cannot express projection (division) as a linear operation. Homogeneous coordinates add an extra coordinate so that projection, rotation, and translation can all be written as matrix multiplication.

- A 2D point (x, y) becomes (x, y, 1) (or any scalar multiple k(x, y, 1)).
- A 3D point (X, Y, Z) becomes (X, Y, Z, 1).

This is the single most exam-important idea in the entire course — nearly every topic (Homography, DLT, Fundamental Matrix, Calibration) relies on homogeneous coordinates to linearize projective transformations.

1.3 Mathematical Formulation

1.3.1 Perspective Projection Equation

For a 3D point (Xc, Yc, Zc) in camera coordinates, the projection onto the image plane at focal length f is:

$$x = f \cdot Xc / Zc, \qquad y = f \cdot Yc / Zc$$

Variables: f = focal length, Zc = depth of the point along the optical axis, (x, y) = projected image-plane coordinates.

Assumption: the pinhole is an ideal point aperture (no lens distortion), and Zc > 0 (point is in front of the camera).

1.3.2 Intrinsic Matrix K

Converts camera coordinates (metric units) to pixel coordinates, accounting for focal length, pixel scaling, skew, and principal-point offset:

$K =$

$$\begin{bmatrix} fx & s & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix}$$

- fx = f·mx, fy = f·my — focal length expressed in pixels (mx, my = pixels per unit length).
- s — skew coefficient (nonzero only for non-rectangular pixels; usually 0).
- (cx, cy) — principal point (where the optical axis intersects the image plane), usually near the image center.

1.3.3 Extrinsic Matrix [R | t]

Converts world coordinates to camera coordinates:

$$[Xc;\ Yc;\ Zc] = R \cdot [Xw;\ Yw;\ Zw] + t$$

- R — 3×3 rotation matrix (camera orientation relative to world), orthonormal ($R^T R = I$, $\det R = 1$).
- t — 3×1 translation vector (camera position relative to world, expressed in the camera frame).

1.3.4 Camera Projection Matrix

Combining intrinsics and extrinsics with homogeneous coordinates gives the full camera model:

$$s \cdot [u;\ v;\ 1] = K\ [R \mid t]\ [Xw;\ Yw;\ Zw;\ 1] = P\ [Xw;\ Yw;\ Zw;\ 1]$$

where P = K[R | t] is the 3×4 camera projection matrix and s is an arbitrary nonzero scale factor (homogeneous scale).

1.4 Matrix Representation & Worked Numerical Example

P(3×4) = K(3×3) · [R | t](3×4)

Let R = I_3 (identity), t = $(0, 0, 5)^T$, and:

$K =$

$$\begin{bmatrix} 1000 & 0 & 320 \\ 0 & 1000 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

Find the pixel coordinates of the world point Xw = (1, 0.5, 10) (camera at world origin, no rotation).

Step 1 — Camera coordinates: Xc = R·Xw + t = (1, 0.5, 10) + (0, 0, 5) = (1, 0.5, 15)

Step 2 — Apply K: K·Xc = (1000(1)+320(15), 1000(0.5)+240(15), 15) = (5800, 4100, 15)

Step 3 — Normalize by the third (homogeneous) coordinate: u = 5800/15 = 386.67, v = 4100/15 = 273.33

Answer: the point projects to pixel (386.67, 273.33).

Python — camera projection (P = K[R|t])

```
import numpy as np

def project_point(K, R, t, Xw):
    """Project a 3D world point to pixel coordinates using P = K[R|t]."""
    Xw = np.asarray(Xw, dtype=float).reshape(3, 1)
    t = np.asarray(t, dtype=float).reshape(3, 1)
    Xc = R @ Xw + t                  # world -> camera coordinates
    x_h = K @ Xc                     # apply intrinsics -> homogeneous pixel coords
    u = x_h[0, 0] / x_h[2, 0]        # normalize by the homogeneous scale
    v = x_h[1, 0] / x_h[2, 0]
    return u, v

K = np.array([[1000, 0, 320],
              [0, 1000, 240],
              [0, 0, 1]], dtype=float)
R = np.eye(3)
t = [0, 0, 5]
Xw = [1, 0.5, 10]

u, v = project_point(K, R, t, Xw)
print(f"Pixel coordinates: ({u:.2f}, {v:.2f})")   # -> (386.67, 273.33)
```

1.5 Formula Sheet

- Perspective projection: $x = f \cdot Xc/Zc$, $y = f \cdot Yc/Zc$
- Full model: $s \cdot [u\ v\ 1]^T = K[R|t][Xw\ Yw\ Zw\ 1]^T$
- $P = K[R|t]$ (rank 3, 3×4, defined up to scale → 11 DoF)
- Homogeneous point: $(x, y) \leftrightarrow (kx, ky, k)$ for any $k \neq 0$

1.6 Applications

Robotics (visual servoing), autonomous vehicles (camera-based perception), medical imaging (endoscopic cameras), AR/VR (pose tracking), image stitching (needs a projection model), surveillance (camera placement/FoV design), 3D reconstruction (basis of triangulation).

1.7 Advantages & Limitations

Aspect	Details
Advantages	Simple, linear (in homogeneous form), well understood, basis for all higher-level geometry
Limitations	Ignores lens distortion; assumes an ideal pinhole (no defocus/blur); infinite depth of field is unrealistic

1.8 Common Mistakes

- Forgetting to convert to homogeneous coordinates before matrix multiplication.

- Forgetting to normalize by the third coordinate after applying P (a very common exam error).
- Confusing image coordinates (metric, origin at center) with pixel coordinates (origin top-left).
- Assuming R need not be orthonormal.

1.9 Key Points / Revision

- P = K[R|t], 11 DoF, rank 3.
- Homogeneous coordinates convert projection into linear matrix multiplication.
- Always divide by the last coordinate to get the actual pixel location.
- Frequently confused: intrinsic vs extrinsic — intrinsic = camera's internal optics (fixed for a camera); extrinsic = camera's pose in the world (changes if the camera moves).

2. Convolution and Filtering

2.1 Introduction

Definition: Convolution and filtering are operations that slide a small matrix (kernel/mask) over an image to produce a modified output image — used for smoothing, sharpening, noise removal, and edge/feature detection.

Importance: Filtering is the foundation of nearly all classical image processing and is conceptually the ancestor of the convolution used in CNNs.

Applications: noise reduction, edge detection, blurring for privacy/aesthetics, medical image denoising, feature extraction for object detection.

2.2 Core Concepts

2.2.1 Convolution vs Correlation

Both slide a kernel over the image and compute a weighted sum, but convolution flips the kernel (180°) before sliding; correlation does not.

$$\text{Correlation: } g(x,y) = \Sigma\Sigma\, h(s,t)\, f(x+s,\, y+t)$$

$$\text{Convolution: } g(x,y) = \Sigma\Sigma\, h(s,t)\, f(x-s,\, y-t)$$

For symmetric kernels (like Gaussian, mean), convolution and correlation give identical results.

Aspect	Convolution	Correlation
Kernel flip	Yes (180°)	No
Commutative	Yes	No
Use case	Filtering theory, CNNs	Template matching
Symmetric kernel result	Same as correlation	Same as convolution

2.2.2 Kernel / Mask

A small matrix (e.g. 3×3) of weights. Placed over each pixel neighborhood, multiplied element-wise with the underlying pixel values, and summed to produce the output pixel.

2.3 Common Filters

Filter	Type	Purpose	Effect on noise
Mean (Box)	Linear, low-pass	Smoothing	Reduces Gaussian noise, blurs edges
Gaussian	Linear, low-pass (weighted)	Smoothing	Better edge preservation than mean
Median	Non-linear	Noise removal	Excellent for salt-and-pepper noise, preserves edges
Sobel	Linear, high-pass	Edge detection (gradient)	Emphasizes horizontal/vertical edges
Prewitt	Linear, high-pass	Edge detection (gradient)	Similar to Sobel, simpler weights
Laplacian	Linear, high-pass	Edge detection (2nd derivative)	Sensitive to noise, detects all-direction edges

2.3.1 Mean vs Gaussian vs Median

Property	Mean Filter	Gaussian Filter	Median Filter
Type	Linear	Linear	Non-linear
Weights	Equal	Gaussian-weighted (center highest)	N/A (uses sorting)
Edge preservation	Poor	Moderate	Good
Best for	Uniform noise	General smoothing	Salt-and-pepper noise
Computation	Fast	Moderate	Slower (sorting)

2.3.2 Standard Kernels

Mean filter (3×3) — each entry scaled by 1/9:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Gaussian (3×3, σ≈1) — each entry scaled by 1/16:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel:

Gx =

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Gy =

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Prewitt:

Gx =

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Gy =

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Laplacian (4-neighbour):

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Sharpening (unsharp masking kernel):

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\text{Gradient magnitude \& direction: } |\nabla f| = \sqrt{(Gx^2 + Gy^2)}, \quad \theta = \tan^{-1}(Gy/Gx)$$

2.4 Solved Numerical Example

Apply a 3×3 mean filter to the center pixel of this image patch: [[10,20,30],[40,50,60],[70,80,90]]

Step 1: Sum all 9 values = 450. Step 2: Divide by 9 → 450/9 = 50.

Result: the center pixel (originally 50) stays 50 (mean filter output = average of neighborhood).

Sobel example on the same patch — Gx = (-1)(10)+(0)(20)+(1)(30)+(-2)(40)+(0)(50)+(2)(60)+(-1)(70)+(0)(80)+(1)(90) = 80

Gy = (-1)(10)+(-2)(20)+(-1)(30)+0+0+0+(1)(70)+(2)(80)+(1)(90) = 240

Gradient magnitude: $|\nabla f| = \sqrt{(80^2 + 240^2)} = \sqrt{64000} \approx 252.98$

2.5 Algorithm: 2D Convolution

Input: image f (M×N), kernel h (k×k). Output: filtered image g.

```
for each pixel (x, y) in f:
    sum = 0
    for s = -a to a:
        for t = -a to a:
            sum += h(s, t) * f(x - s, y - t)   # convolution (flip)
    g(x, y) = sum
```

Time Complexity: $O(M \cdot N \cdot k^2)$ for direct convolution; O(MN log(MN)) via FFT for large kernels.

Padding note: border pixels need zero-padding, replication, or reflection to keep the output size equal to the input size.

Python — 2D convolution, mean/Gaussian/median filters, and Sobel gradient

```
import numpy as np

def convolve2d(f, h, padding="zero"):
    """Direct 2D convolution (kernel is flipped 180 degrees, as in the definition)."""
    h = np.flip(h)                      # flip kernel -> turns correlation into convolution
    kh, kw = h.shape
    ph, pw = kh // 2, kw // 2
    if padding == "zero":
        fp = np.pad(f, ((ph, ph), (pw, pw)), mode="constant")
    elif padding == "reflect":
        fp = np.pad(f, ((ph, ph), (pw, pw)), mode="reflect")
    else:
        fp = np.pad(f, ((ph, ph), (pw, pw)), mode="edge")   # replication
```

```
    out = np.zeros_like(f, dtype=float)
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            region = fp[y:y + kh, x:x + kw]
            out[y, x] = np.sum(region * h)
    return out

def mean_filter(f, size=3):
    kernel = np.ones((size, size)) / (size * size)
    return convolve2d(f, kernel)

def gaussian_kernel(size=3, sigma=1.0):
    ax = np.arange(-(size // 2), size // 2 + 1)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
    return kernel / kernel.sum()

def median_filter(f, size=3):
    ph = size // 2
    fp = np.pad(f, ph, mode="edge")
    out = np.zeros_like(f, dtype=float)
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            region = fp[y:y + size, x:x + size]
            out[y, x] = np.median(region)
    return out

def sobel_gradient(f):
    Gx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=float)
    Gy = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=float)
    gx = convolve2d(f, Gx)
    gy = convolve2d(f, Gy)
    magnitude = np.sqrt(gx**2 + gy**2)
    direction = np.arctan2(gy, gx)
    return magnitude, direction

# Worked example from the notes
patch = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]], dtype=float)
print("Mean filter center:", mean_filter(patch)[1, 1])          # -> 50.0
mag, _ = sobel_gradient(patch)
print("Sobel gradient magnitude (center):", round(mag[1, 1], 2))  # -> 252.98
```

2.6 Formula Sheet

- Convolution: $g(x,y) = \Sigma\, h(s,t)\, f(x-s, y-t)$
- Gradient magnitude: $|\nabla f| = \sqrt{(Gx^2 + Gy^2)}$, direction $\theta = \tan^{-1}(Gy/Gx)$
- Mean filter = uniform average; Gaussian = weighted average with $(1/2\pi\sigma^2)\cdot e^{\wedge}(-(x^2+y^2)/2\sigma^2)$

2.7 Applications

Robotics (obstacle edge detection), autonomous vehicles (lane detection via Sobel), medical imaging (denoising MRI/CT), AR/VR (feature detection), image stitching (feature preprocessing), surveillance (noise removal in low light).

2.8 Advantages & Limitations

Advantages: simple, fast, well understood, foundation for deep learning. Limitations: linear filters blur edges; fixed kernels can't adapt to local content; sensitive to kernel size choice.

2.9 Common Mistakes

- Confusing convolution (flipped kernel) with correlation (not flipped) — frequently asked to distinguish in exams.
- Forgetting kernel normalization (dividing by the sum of weights).
- Using a mean filter when a median filter is appropriate for salt-and-pepper noise.
- Forgetting to handle image borders.

2.10 Key Points / Revision

- Convolution flips the kernel; correlation doesn't; identical for symmetric kernels.
- Median filter is best for impulse noise; Gaussian is best for general smoothing while preserving edges better than mean.
- Sobel/Prewitt = first-derivative (gradient) edge detectors; Laplacian = second-derivative (zero-crossing) edge detector.

3. Image Enhancement

3.1 Introduction

Definition: Image enhancement improves the visual quality or interpretability of an image (contrast, brightness, sharpness) without adding new information, typically via point processing (pixel-by-pixel intensity transformations).

Applications: medical imaging (X-ray/CT contrast enhancement), satellite imaging, low-light photography, surveillance footage clean-up.

3.2 Core Concepts

3.2.1 Contrast Stretching

Linearly expands the range of intensity values to span the full available range, improving contrast in low-contrast images.

$$s = (r - r_min) \times (L - 1) / (r_max - r_min)$$

where r = input intensity, s = output intensity, r_min, r_max = min/max intensities in the input, L−1 = max possible intensity (255 for 8-bit).

3.2.2 Gamma Correction (Power-Law Transformation)

$$s = c \cdot r\hat{}\gamma$$

- γ < 1: brightens dark regions (expands dark tones) — used to correct for display/sensor non-linearity.
- γ > 1: darkens the image, increases contrast in bright regions.
- c: scaling constant (usually 1 if r is normalized to [0, 1]).

3.2.3 Log Transformation

$$s = c \cdot log(1 + r)$$

Expands dark pixel values while compressing bright ones — useful for images with a large dynamic range (e.g., Fourier spectra).

3.2.4 Histogram Equalization

Redistributes pixel intensities so the output histogram is approximately uniform, maximizing global contrast. (Full derivation in Topic 12.)

3.2.5 CLAHE (Contrast Limited Adaptive Histogram Equalization)

Applies histogram equalization locally (on small tiles) instead of globally, with a clip limit to prevent noise amplification — better than plain HE for images with varying local lighting.

Aspect	Global HE	CLAHE
Scope	Whole image	Local tiles
Noise amplification	High (in flat regions)	Limited (clip threshold)
Computation	Fast	Slower
Use case	Uniform lighting	Non-uniform lighting (medical images)

3.2.6 Noise Reduction

Achieved via spatial filters (mean, Gaussian, median — see Topic 2) or frequency-domain low-pass filtering.

3.3 Solved Numerical Example — Contrast Stretching

Given an image with intensities ranging from r_min = 50 to r_max = 200, 8-bit image (L = 256). Find the stretched value of a pixel with r = 100.

$$s = (100 - 50) \times 255 / (200 - 50) = 50 \times 1.7 = 85$$

Answer: the pixel intensity 100 maps to 85 after contrast stretching.

3.3.1 Gamma Correction Example

For r = 0.5 (normalized), γ = 2.2, c = 1: s = (0.5)^2.2 = 0.217 — the mid-gray pixel darkens significantly, typical of CRT gamma decoding.

Python — contrast stretching, gamma correction, log transform

```
import numpy as np

def contrast_stretch(r, r_min, r_max, L=256):
    """Linear contrast stretching. r can be a scalar or a numpy array."""
    r = np.asarray(r, dtype=float)
    s = (r - r_min) * (L - 1) / (r_max - r_min)
    return np.clip(s, 0, L - 1)

def gamma_correction(r, gamma, c=1.0):
    """r assumed normalized to [0, 1]. gamma < 1 brightens, gamma > 1 darkens."""
    r = np.asarray(r, dtype=float)
    return c * np.power(r, gamma)

def log_transform(r, c=None, L=256):
    """Log transform; c chosen so the max input maps to L-1 if not given."""
    r = np.asarray(r, dtype=float)
    if c is None:
        c = (L - 1) / np.log(1 + r.max())
    return c * np.log(1 + r)

# Worked examples from the notes
print(contrast_stretch(100, 50, 200, L=256))   # -> 85.0
print(round(gamma_correction(0.5, 2.2), 3))    # -> 0.217
```

3.4 Formula Sheet

- Contrast stretch: s = (r − r_min)(L−1)/(r_max − r_min)
- Gamma: s = c·r^γ
- Log: s = c·log(1+r)
- CLAHE = local HE + clip limit

3.5 Applications

Medical imaging (enhancing tumor visibility in X-rays), autonomous vehicles (enhancing low-light camera feed), surveillance (night footage enhancement), AR/VR (dynamic range adjustment), robotics (preprocessing for feature detection).

3.6 Advantages & Limitations

Advantages: simple, fast, real-time capable, improves human/algorithm interpretability. Limitations: point operations ignore spatial context; global HE can over-amplify noise; gamma choice is often empirical.

3.7 Common Mistakes

- Confusing gamma correction direction ($\gamma < 1$ brightens, not darkens).
- Forgetting to clip output values to the valid range $[0, L-1]$.
- Applying global HE to images with non-uniform lighting (CLAHE is needed instead).
- Mixing up log transform's dark-region expansion with gamma's flexible curve.

3.8 Key Points / Revision

- Contrast stretching = linear; Gamma = power-law (non-linear, tunable); Log = compresses high dynamic range.
- CLAHE > global HE for medical/satellite images with local lighting variation.
- Frequently confused: Histogram Equalization vs Histogram Matching — see the Topic 12 comparison table.

4. Depth Estimation

4.1 Introduction

Definition: Depth estimation recovers the distance (Z) of scene points from the camera, typically using two (or more) images captured from different viewpoints (stereo vision).

Importance: A single 2D image loses depth information (Topic 1 shows projection divides by Z). Depth estimation recovers this lost dimension, enabling 3D understanding.

Applications: autonomous driving (obstacle distance), robotics (grasping, navigation), AR/VR (occlusion handling), 3D reconstruction, medical imaging (endoscopic depth).

4.2 Core Concepts

4.2.1 Stereo Vision Setup

Two cameras (left, right) separated by a baseline B, both viewing the same scene. A 3D point projects to different horizontal positions in the two images — this difference is the disparity.

4.2.2 Disparity

$$d = x_left - x_right$$

Intuition: nearby objects shift a lot between the two views (large disparity); far objects barely shift (small disparity) — the same principle as human binocular vision.

4.2.3 Triangulation

Given corresponding points in two calibrated views, triangulation computes the 3D position by intersecting the two viewing rays.

4.3 Mathematical Formulation

4.3.1 Depth-Disparity Relation

For a simple rectified stereo rig (parallel cameras, same focal length f, baseline B):

$$Z = f \cdot B \,/\, d$$

Variables: Z = depth, f = focal length (pixels), B = baseline (distance between camera centers), d = disparity (pixels).

Assumption: cameras are rectified so that corresponding points lie on the same horizontal scanline (see Topic 8) — otherwise disparity must be computed along epipolar lines, not a simple horizontal difference.

Key insight (inverse relationship): Z ∝ 1/d — depth estimation is much more precise for near objects than far ones (small disparity errors cause huge depth errors at large Z).

4.4 Solved Numerical Example

A stereo rig has focal length f = 700 pixels, baseline B = 0.12 m. A feature point has disparity d = 28 pixels. Find the depth.

$$Z = (700 \times 0.12) \,/\, 28 = 84 \,/\, 28 = 3\ m$$

Answer: the point is 3 meters from the camera.

Follow-up (sensitivity): if disparity estimation has an error of ±1 pixel at d = 28: Z' = 84/27 ≈ 3.11 m or 84/29 ≈ 2.90 m — an error of about ±0.10–0.11 m, illustrating why far-away disparity errors matter more.

4.5 Algorithm: Stereo Depth Estimation

Input: rectified stereo image pair (left, right). Output: dense depth map.

```
1. Rectify stereo pair (align epipolar lines horizontally)
2. For each pixel in left image:
     search along the same row in right image
     find best matching pixel (block matching / SSD / SAD / semi-global matching)
     compute disparity d = x_left - x_right
3. Convert disparity map to depth map: Z = f*B/d
4. (Optional) Post-process: hole filling, smoothing
```

Time Complexity: O(W·H·D) where D = disparity search range, for naive block matching.

Python — disparity-to-depth conversion and simple block-matching stereo

```
import numpy as np

def depth_from_disparity(f, B, d):
    """Z = f*B/d. d=0 (or near-zero) means infinite/undefined depth."""
    d = np.asarray(d, dtype=float)
    with np.errstate(divide="ignore"):
        Z = np.where(d > 0, (f * B) / d, np.inf)
    return Z

def block_match_disparity(left, right, max_disparity=32, block_size=5):
    """Very simple SAD block-matching stereo (rectified images assumed)."""
    h, w = left.shape
    half = block_size // 2
    disparity_map = np.zeros((h, w), dtype=float)

    for y in range(half, h - half):
        for x in range(half, w - half):
            left_block = left[y - half:y + half + 1, x - half:x + half + 1]
            best_offset, best_cost = 0, np.inf
            for d in range(0, min(max_disparity, x - half)):
                xr = x - d
                right_block = right[y - half:y + half + 1, xr - half:xr + half + 1]
                cost = np.sum(np.abs(left_block.astype(float) - right_block.astype(float)))
                if cost < best_cost:
                    best_cost, best_offset = cost, d
            disparity_map[y, x] = best_offset
    return disparity_map

# Worked example
Z = depth_from_disparity(f=700, B=0.12, d=28)
print(f"Depth: {Z:.2f} m")   # -> 3.00 m
```

4.6 Formula Sheet

- Depth-disparity: Z = fB/d
- Disparity: d = x_left − x_right
- Depth is inversely proportional to disparity — near objects have large disparity, high depth precision.

4.7 Applications

Autonomous vehicles (LIDAR-free depth sensing), robotics (obstacle avoidance, grasping), AR/VR (scene occlusion, hand tracking), 3D reconstruction (point clouds), medical imaging (stereo endoscopy), surveillance (people distance estimation).

4.8 Advantages & Limitations

Aspect	Details
Advantages	Passive (no special hardware beyond 2 cameras), works outdoors, dense depth possible
Limitations	Fails on textureless/low-texture regions, occlusions cause missing disparity, needs accurate calibration & rectification, computationally expensive for dense matching

4.9 Common Mistakes

- Forgetting that Z = fB/d only holds for rectified images.
- Using pixel disparity without converting focal length to the same pixel units.
- Confusing disparity (pixel shift) with depth (metric distance) — inversely related, not equal.
- Ignoring occlusion regions (no valid match exists) when computing dense depth maps.

4.10 Key Points / Revision

- Z = fB/d is the single most important formula — memorize units (pixels for f, d; meters for B, Z).
- Larger baseline → better depth accuracy at large distances, but harder to match correspondences (larger disparity search range, more occlusion).
- Triangulation requires calibrated cameras (intrinsics + extrinsics known, Topic 11).

5. Fundamental Matrix

5.1 Introduction

Definition: The Fundamental Matrix F (3×3, rank 2) encodes the epipolar geometry between two uncalibrated cameras — it relates corresponding points in a stereo pair without requiring knowledge of intrinsic/extrinsic parameters.

Importance: F lets us constrain the search for correspondences to a single line (epipolar line) instead of the whole image, and is the starting point for uncalibrated stereo reconstruction.

Applications: structure-from-motion, image stitching (with additional constraints), stereo correspondence search, 3D reconstruction from uncalibrated cameras.

5.2 Core Concepts

5.2.1 The Epipolar Constraint

For corresponding points x (left image) and x' (right image) of the same 3D point:

$$x'^T F x = 0$$

Intuition: given a point in the left image, its corresponding point in the right image is not free to be anywhere — it is constrained to lie on a specific line (the epipolar line), because both must correspond to the same 3D ray. F encodes exactly this line for every point.

5.2.2 Properties of F

- Size: 3×3
- Rank 2 (singular, det F = 0) — the epipole maps every line through it to a single point, forcing rank deficiency.
- 7 Degrees of Freedom: 9 entries − 1 (scale ambiguity) − 1 (rank-2 constraint) = 7.
- Defined only up to scale (like all homogeneous quantities).
- F e = 0 and F^T e' = 0, where e, e' are the epipoles.

5.3 Mathematical Formulation

5.3.1 Relation to Essential Matrix

$$F = K'^{-T} E K^{-1}$$

where K, K' are intrinsic matrices of the two cameras and E is the Essential matrix (Topic 6). F is simply E expressed in pixel coordinates rather than normalized camera coordinates.

5.3.2 The Eight-Point Algorithm (derivation of F)

Each point correspondence (u,v) ↔ (u',v') gives one linear equation in the 9 unknown entries of F:

$$u'u\, f11 + u'v\, f12 + u'\, f13 + v'u\, f21 + v'v\, f22 + v'\, f23 + u\, f31 + v\, f32 + f33 = 0$$

Stacking ≥ 8 correspondences gives Af = 0, solved via SVD (smallest singular vector of A); the resulting F is then corrected to rank 2 by zeroing its smallest singular value (SVD again).

5.4 Matrix Representation

F =

$$\begin{bmatrix} f11 & f12 & f13 \\ f21 & f22 & f23 \\ f31 & f32 & f33 \end{bmatrix}$$

rank(F) = 2

5.5 Solved Numerical Example

Given a point in the left image $x = (1,1,1)^T$ and:

F =

$$\begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & -3 \\ -2 & 3 & 0 \end{bmatrix}$$

find the epipolar line in the right image.

Step 1 — Compute l' = Fx = (0(1)+(-1)(1)+2(1), 1(1)+0(1)+(-3)(1), -2(1)+3(1)+0(1)) = (1, -2, 1)

Step 2 — Interpret: the epipolar line in the right image is 1·u' − 2·v' + 1 = 0, i.e. u' − 2v' + 1 = 0.

Verification: any correct correspondence x' = (u',v',1) must satisfy $x'^T l' = 0$ — this is how F narrows the correspondence search from a 2D image to a 1D line.

5.6 Algorithm: Eight-Point Algorithm

Input: ≥ 8 point correspondences (xi, x'i). Output: Fundamental matrix F.

```
1. Normalize point coordinates (translate centroid to origin, scale to avg dist sqrt(2)) —
Hartley normalization
2. Build matrix A (n x 9) from correspondences
3. Solve Af = 0 via SVD -> f = last column of V (smallest singular value)
4. Reshape f into 3x3 matrix F
5. Enforce rank-2: SVD(F) = U*diag(s1,s2,s3)*V^T -> set s3=0 -> F = U*diag(s1,s2,0)*V^T
6. Denormalize F
```

Time Complexity: O(n) for building A with n points; SVD is $O(9^3)$ (constant, since A is n×9).

Python — Eight-Point Algorithm for the Fundamental Matrix

```
import numpy as np

def normalize_points(pts):
    """Hartley normalization: centroid at origin, average distance sqrt(2)."""
    centroid = pts.mean(axis=0)
    shifted = pts - centroid
    mean_dist = np.mean(np.sqrt(np.sum(shifted**2, axis=1)))
    scale = np.sqrt(2) / mean_dist
    T = np.array([[scale, 0, -scale * centroid[0]],
                  [0, scale, -scale * centroid[1]],
                  [0, 0, 1]])
    pts_h = np.hstack([pts, np.ones((pts.shape[0], 1))])
    pts_norm = (T @ pts_h.T).T
    return pts_norm[:, :2], T

def eight_point_algorithm(pts1, pts2):
    """
    pts1, pts2: Nx2 arrays of corresponding image points (N >= 8).
    Returns the 3x3 rank-2 Fundamental matrix F.
    """
    pts1n, T1 = normalize_points(pts1)
```

```
    pts2n, T2 = normalize_points(pts2)

    n = pts1.shape[0]
    A = np.zeros((n, 9))
    for i in range(n):
        u, v = pts1n[i]
        up, vp = pts2n[i]
        A[i] = [up*u, up*v, up, vp*u, vp*v, vp, u, v, 1]

    # Solve Af = 0 via SVD -> smallest singular vector
    _, _, Vt = np.linalg.svd(A)
    F = Vt[-1].reshape(3, 3)

    # Enforce rank-2 constraint
    U, S, Vt2 = np.linalg.svd(F)
    S[2] = 0
    F = U @ np.diag(S) @ Vt2

    # Denormalize
    F = T2.T @ F @ T1
    return F / F[2, 2]

def epipolar_line(F, x):
    """Epipolar line l' = F x (in the right image) for a point x in the left image."""
    return F @ np.append(x, 1)

# Worked example from the notes
F = np.array([[0, -1, 2], [1, 0, -3], [-2, 3, 0]], dtype=float)
l_prime = epipolar_line(F, [1, 1])
print("Epipolar line l':", l_prime)   # -> [1, -2, 1]  =>  u' - 2v' + 1 = 0
```

5.7 Formula Sheet

- Epipolar constraint: $x'^{T} F x = 0$
- $F = K'^{-T} E K^{-1}$
- rank(F) = 2, 7 DoF
- Epipolar line in right image: $l' = Fx$; in left image: $l = F^{T}x'$
- $F e = 0$, $F^{T} e' = 0$

5.8 Fundamental Matrix vs Essential Matrix

Property	Fundamental Matrix (F)	Essential Matrix (E)
Cameras	Uncalibrated (pixel coords)	Calibrated (normalized coords)
Size	3×3	3×3
Rank	2	2
DoF	7	5
Relation	$F = K'^{-T}EK^{-1}$	$E = K'^{T}FK$
Constraint	$x'^{T}Fx = 0$	$\hat{X}'^{T}E\hat{X} = 0$ (normalized coords)
Singular values	Arbitrary (rank 2)	Two equal, one zero

5.9 Applications

Structure-from-motion, stereo correspondence, uncalibrated 3D reconstruction, robotics SLAM (visual odometry), image stitching validation.

5.10 Advantages & Limitations

Advantages: works without camera calibration; reduces correspondence search to 1D. Limitations: sensitive to noise/outliers in correspondences (needs RANSAC, Topic 10); degenerate for planar scenes (use Homography instead); scale/projective ambiguity remains (cannot recover metric structure without calibration).

5.11 Common Mistakes

- Forgetting the rank-2 enforcement step (raw SVD solution of Af = 0 is not guaranteed rank 2).
- Confusing which image the epipolar line Fx belongs to (it's the line in the other image).
- Not normalizing coordinates before running the eight-point algorithm (causes numerical instability).
- Confusing F (uncalibrated, 7 DoF) with E (calibrated, 5 DoF).

5.12 Key Points / Revision

- F needs only pixel correspondences — no calibration required.
- Minimum 8 points for the linear algorithm (7 points possible with non-linear constraint enforcement).
- Always enforce rank-2 via SVD truncation.
- F encodes epipolar lines; combined with calibration gives E, which further gives relative pose (R, t).

6. Epipolar Geometry

6.1 Introduction

Definition: Epipolar geometry is the intrinsic projective geometry between two views of the same scene — it describes the geometric relationship between corresponding points in a stereo image pair, independent of scene structure.

Importance: it is the geometric foundation underlying the Fundamental and Essential matrices, stereo rectification, and depth estimation.

Applications: stereo vision, 3D reconstruction, visual SLAM, image rectification.

6.2 Core Concepts

6.2.1 The Epipolar Geometry Setup

- Baseline: the line joining the two camera centers C and C'.
- Epipole (e, e'): the point where the baseline intersects each image plane — it is the image, in one camera, of the other camera's center.
- Epipolar plane: the plane formed by the 3D point P and the two camera centers C, C'.
- Epipolar line: the intersection of the epipolar plane with an image plane. Every point in the left image has a corresponding epipolar line in the right image.

6.2.2 Key Idea (why it matters for correspondence search)

For any 3D point P, its epipolar plane is determined by P, C, C'. As P varies, all epipolar lines in a given image pass through that image's epipole — this is why the search for corresponding points reduces from a 2D image search to a 1D line search.

6.2.3 Essential Matrix

The calibrated counterpart of the Fundamental matrix — it encodes the epipolar geometry in normalized camera coordinates:

$$E = [t]_\times R$$

where R, t are the rotation and translation between the two camera frames, and [t]× is the skew-symmetric matrix form of t (so that [t]× v = t × v).

[t]× =

$$\begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

Constraint: $\hat{x}'^{T} E \hat{x} = 0$ where $\hat{x} = K^{-1}x$ (normalized image coordinates).

Properties of E: rank 2, exactly two equal non-zero singular values (and one zero), 5 DoF (3 for rotation, 2 for translation direction — translation magnitude is unrecoverable from images alone).

6.3 Point Correspondence via Epipolar Geometry

- Compute epipolar line l' = Fx in the right image for a point x in the left image.
- The correct match x' must lie on l' — search is restricted to this line only.

- If cameras are rectified, all epipolar lines become horizontal, so the search reduces to scanning a single row (see Topic 8).

Python — Essential matrix from R, t, and the epipolar constraint check

```
import numpy as np

def skew(t):
    """Skew-symmetric matrix [t]x such that [t]x @ v == cross(t, v)."""
    tx, ty, tz = t
    return np.array([[0, -tz, ty],
                     [tz, 0, -tx],
                     [-ty, tx, 0]])

def essential_matrix(R, t):
    """E = [t]x R"""
    return skew(t) @ R

def epipolar_constraint_error(F, x, x_prime):
    """Should be ~0 for a correct correspondence: x'^T F x = 0"""
    x_h = np.append(x, 1)
    xp_h = np.append(x_prime, 1)
    return xp_h @ F @ x_h

R = np.eye(3)
t = np.array([1.0, 0.0, 0.0])
E = essential_matrix(R, t)
print("Essential matrix:\n", E)
```

6.4 Formula Sheet

- Epipolar constraint (uncalibrated): $x'^{T} F x = 0$
- Epipolar constraint (calibrated): $\hat{x}'^{T} E \hat{x} = 0$
- $E = [t]_{\times} R$
- $F = K'^{-T} E K^{-1}$
- Epipoles: $F e = 0$, $F^{T} e' = 0$
- All epipolar lines in an image pass through that image's epipole.

6.5 Applications

Stereo correspondence search, structure-from-motion, visual odometry/SLAM, autonomous vehicles (multi-camera depth), 3D reconstruction, image rectification pipeline.

6.6 Advantages & Limitations

Advantages: reduces 2D search to 1D, purely geometric (no scene content needed), works for any relative camera pose. Limitations: degenerates when camera centers coincide (pure rotation — no baseline); breaks down for points near/at infinity in some configurations; epipole may lie outside the visible image.

6.7 Common Mistakes

- Thinking the epipole is the principal point — it is not; it is the projection of the other camera's center.
- Forgetting that epipolar lines converge at the epipole, not that they are parallel (except after rectification).
- Mixing up E (calibrated, normalized coords) and F (uncalibrated, pixel coords).

- Forgetting E has only 5 DoF (not 7 like F) due to the additional constraint of two equal singular values.

6.8 Key Points / Revision

- Epipolar geometry is purely a function of camera pose, not scene content.
- F (pixel space) and E (normalized/calibrated space) both encode the same geometry.
- Rectification (Topic 8) transforms epipolar lines to be horizontal and identical in both images — dramatically simplifying stereo matching.

7. Homography

7.1 Introduction

Definition: A homography is a 3×3 invertible matrix H that describes a projective transformation between two planes — e.g., mapping points on a planar scene (or a camera rotating about its optical center) from one image to another.

Importance: unlike the Fundamental matrix (general 3D scenes), homography applies specifically when scene points are coplanar, or when the camera only rotates (no translation) — very common in practice (image stitching, document scanning).

Applications: panorama/image stitching, planar object tracking (AR markers), perspective correction (document scanning), bird's-eye view generation for autonomous driving.

7.2 Core Concepts

7.2.1 Planar / Perspective Transformation

$$[x'; y'; 1] \sim H\,[x; y; 1]$$

The '∼' denotes equality up to scale — after multiplying, we must divide by the third (homogeneous) coordinate to recover actual pixel coordinates.

7.2.2 When does a Homography Apply?

- The scene is (approximately) planar (e.g., a wall, floor, document page), OR
- The camera undergoes pure rotation (no translation) between the two views.

If neither condition holds, points at different depths will not be consistently mapped by a single H (parallax appears) — Fundamental matrix / epipolar geometry is needed instead.

7.3 Mathematical Formulation

$H =$

$$\begin{bmatrix} h11 & h12 & h13 \\ h21 & h22 & h23 \\ h31 & h32 & h33 \end{bmatrix}$$

3×3, defined up to scale → 8 Degrees of Freedom (9 entries − 1 for scale).

Each point correspondence gives 2 independent equations, so 4 point correspondences (no 3 collinear) are the minimum needed to solve for H.

Expanding and eliminating scale:

$$x' = (h11\,x + h12\,y + h13) \,/\, (h31\,x + h32\,y + h33)$$

$$y' = (h21\,x + h22\,y + h23) \,/\, (h31\,x + h32\,y + h33)$$

Rearranged into linear form (used for solving via DLT, Topic 9):

$$x'(h31\,x + h32\,y + h33) = h11\,x + h12\,y + h13$$

$$y'(h31\,x + h32\,y + h33) = h21\,x + h22\,y + h23$$

7.4 Matrix Representation & Worked Numerical Example

Suppose (a pure scale-by-2 + translation homography):

$$H = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Find where point (1,1) in Image A maps to in Image B.

Step 1 — Homogeneous form: $x = (1,1,1)^T$. Step 2 — Apply H: $Hx = (3,3,1)^T$. Step 3 — Normalize (÷1, already normalized): $x' = 3$, $y' = 3$.

Answer: point (1,1) maps to (3,3) in Image B.

Second point, to show normalization matters — apply:

$$H' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.5 & 0 & 1 \end{bmatrix}$$

to (2,2): $H'x = (2,2,2)^T \Rightarrow x'=2/2=1$, $y'=2/2=1$. This shows how the projective (bottom-row) terms create non-affine warping.

7.5 Algorithm: Estimating Homography

(overview; full derivation in DLT, Topic 9). Input: ≥ 4 point correspondences between two images. Output: 3×3 matrix H.

```
1. Collect >=4 point correspondences (x_i, x_i')
2. Build linear system Ah = 0 using the two equations per correspondence
3. Solve via SVD (DLT) -> h = smallest singular vector
4. Reshape h into 3x3 H, normalize so h33 = 1 (optional)
5. (If >4 points / noisy data) use RANSAC to reject outliers, then re-estimate H
```

Time Complexity: O(n) to build the system with n points; SVD cost is constant (8×9 matrix).

Python — apply a homography and estimate one from correspondences (DLT)

```
import numpy as np

def apply_homography(H, pt):
    """Map a 2D point through H, with proper homogeneous normalization."""
    x_h = H @ np.array([pt[0], pt[1], 1.0])
    return x_h[0] / x_h[2], x_h[1] / x_h[2]

def estimate_homography(pts1, pts2):
    """
    DLT estimation of H from >=4 correspondences.
    pts1, pts2: Nx2 arrays, pts2 = H * pts1 (up to scale).
    """
    n = pts1.shape[0]
    A = np.zeros((2 * n, 9))
    for i in range(n):
        x, y = pts1[i]
        xp, yp = pts2[i]
        A[2*i]   = [-x, -y, -1, 0, 0, 0, xp*x, xp*y, xp]
        A[2*i+1] = [0, 0, 0, -x, -y, -1, yp*x, yp*y, yp]
```

```
    _, _, Vt = np.linalg.svd(A)
    H = Vt[-1].reshape(3, 3)
    return H / H[2, 2]

# Worked example
H = np.array([[2, 0, 1], [0, 2, 1], [0, 0, 1]], dtype=float)
print(apply_homography(H, (1, 1)))   # -> (3.0, 3.0)
```

7.6 Formula Sheet

- x' ∼ Hx (homogeneous, up to scale)
- H: 3×3, 8 DoF, minimum 4 point correspondences
- Applies to planar scenes or pure camera rotation
- Always normalize by the 3rd coordinate after multiplying

7.7 Applications

Image stitching/panoramas, AR marker tracking (planar fiducials), document/perspective rectification (scanning apps), bird's-eye-view transform for autonomous driving lane detection, video stabilization.

7.8 Advantages & Limitations

Advantages: simple linear model, works well for planar/rotation-only scenarios, only needs 4 correspondences. Limitations: fails for non-planar scenes with camera translation (parallax breaks the model); sensitive to outlier correspondences (needs RANSAC); large perspective distortion can amplify small correspondence errors.

7.9 Common Mistakes

- Applying homography to a general 3D scene with camera translation (only valid for planar/rotation cases).
- Forgetting to normalize by the homogeneous scale factor after transformation.
- Using only 3 (collinear or too few) correspondences — need 4 non-collinear points minimum.
- Confusing homography (plane-to-plane) with the Fundamental matrix (general 3D scene, epipolar constraint).

7.10 Key Points / Revision

- 8 DoF, needs 4 point correspondences minimum.
- Valid only for planar scenes or pure-rotation camera motion.
- Solved via DLT + SVD; refined using RANSAC when correspondences are noisy.
- Frequently confused with the Fundamental matrix — Homography = 2D-2D plane mapping; F = 2D-2D general epipolar constraint (3D scene).

8. Rectification

8.1 Introduction

Definition: Stereo rectification is the process of transforming (warping) a pair of stereo images so that corresponding epipolar lines become horizontal and aligned on the same row in both images.

Importance: it converts the general 2D correspondence search along arbitrary epipolar lines into a simple 1D horizontal search, dramatically simplifying and speeding up stereo matching and depth estimation.

Applications: stereo depth estimation, autonomous vehicle stereo cameras, robotic stereo heads, structured-light 3D scanning pre-processing.

8.2 Core Concepts

8.2.1 The Problem Before Rectification

In a general (non-rectified) stereo pair, epipolar lines are slanted and converge at the epipole — searching for correspondences means searching along a differently-angled line for every point.

8.2.2 Stereo Rectification Goal

Apply a homography (Topic 7) to each image independently such that:

- Epipolar lines become horizontal.
- Corresponding epipolar lines in the left and right images have the same row (y-coordinate).
- Effectively, this simulates both cameras being coplanar with parallel optical axes and aligned image planes (as if the epipoles were moved to infinity).

8.2.3 Horizontal Epipolar Lines

After rectification, for any point in the left image at row v, its match in the right image is guaranteed to also be at row v — search reduces to scanning that single row for the best-matching pixel (used for computing disparity, Topic 4).

8.3 Mathematical Formulation

Rectification computes a pair of homographies H1 (left image) and H2 (right image) such that the epipoles are mapped to infinity along the horizontal direction, i.e., epipole e → $(1,0,0)^T$ in homogeneous coordinates.

General approach (calibrated case):

- Compute a rotation R_rect that aligns the baseline with the new X-axis.
- Apply R_rect to both cameras' rotation matrices to get new, identical orientations.
- New camera matrices K[R_rect|t] produce parallel, row-aligned image planes.

Uncalibrated case: rectifying homographies H1, H2 are estimated directly from the Fundamental matrix F (and epipoles), typically via Hartley's algorithm, mapping each epipole to the point at infinity along the x-axis.

8.4 Image Alignment Pipeline

World Point → Left Camera → Left Image (unrectified) → apply H1 → Left Image (rectified)

World Point → Right Camera → Right Image (unrectified) → apply H2 → Right Image (rectified)

After rectification, corresponding points share the same row (v-coordinate).

8.5 Algorithm: Stereo Rectification (calibrated case)

Input: camera matrices K1[R1|t1], K2[R2|t2] (or F if uncalibrated). Output: rectifying homographies H1, H2, rectified image pair.

```
1. Compute baseline direction: e1 = (C2 - C1) / ||C2 - C1|| (new X-axis)
2. Compute e2 orthogonal to e1 and original optical axis (new Y-axis)
3. Compute e3 = e1 x e2 (new Z-axis)
4. R_rect = [e1^T; e2^T; e3^T]
5. New projection matrices: P1' = K[R_rect | -R_rect*C1], P2' = K[R_rect | -R_rect*C2]
6. H1 = P1'(1:3,1:3) * inv(P1(1:3,1:3))  (similarly H2)
7. Warp both images using H1, H2
```

Python — computing the rectifying rotation and homographies

```
import numpy as np

def rectifying_rotation(C1, C2, optical_axis):
    """Build the shared rectified rotation matrix R_rect from camera centers."""
    e1 = (C2 - C1) / np.linalg.norm(C2 - C1)              # new X-axis: along baseline
    e2 = np.cross(optical_axis, e1)
    e2 = e2 / np.linalg.norm(e2)                           # new Y-axis: orthogonal to e1 &
old axis
    e3 = np.cross(e1, e2)                                  # new Z-axis
    R_rect = np.vstack([e1, e2, e3])
    return R_rect

def rectifying_homography(K, R_old, R_rect):
    """H maps the old image to the rectified image for one camera."""
    return K @ R_rect @ np.linalg.inv(R_old) @ np.linalg.inv(K)

# Example: two cameras with the same K, baseline along X
K = np.array([[700, 0, 320], [0, 700, 240], [0, 0, 1]], dtype=float)
C1 = np.array([0.0, 0.0, 0.0])
C2 = np.array([0.12, 0.0, 0.0])          # 12 cm baseline
R_rect = rectifying_rotation(C1, C2, optical_axis=np.array([0, 0, 1.0]))
H1 = rectifying_homography(K, np.eye(3), R_rect)
print("H1:\n", H1)
```

8.6 Formula Sheet

- Goal: epipoles → point at infinity along the horizontal axis
- After rectification: y_left = y_right for all correspondences
- Depth from rectified disparity: Z = fB/d (Topic 4) — only valid after rectification
- Rectification uses a pair of homographies H1, H2

8.7 Applications

Stereo depth cameras (autonomous vehicles, robots), 3D reconstruction pipelines, structured light scanning, real-time stereo matching hardware (FPGAs use rectified rows for pipeline efficiency).

8.8 Advantages & Limitations

Advantages: reduces 2D correspondence search to 1D — a massive speed-up; simplifies the disparity-to-depth formula. Limitations: requires accurate calibration (or accurate F) — errors cause vertical misalignment and matching failures; rectification can introduce image resampling artifacts and reduce field of view/resolution near image borders.

8.9 Common Mistakes

- Using the simple Z = fB/d formula on unrectified images (invalid).
- Assuming rectification is optional for stereo matching — without it, matching is far more expensive and error-prone.
- Forgetting that rectification requires both cameras to end up with the same (virtual) orientation and identical focal lengths in the rectified frame.

8.10 Key Points / Revision

- Purpose: align epipolar lines horizontally to enable a fast 1D disparity search.
- Achieved via a pair of homographies applied to each image.
- A prerequisite step for practical dense stereo depth estimation (Topic 4).

9. Direct Linear Transformation (DLT)

9.1 Introduction

Definition: DLT is a general algebraic method for solving systems of 'linear-up-to-scale' equations — most commonly used to estimate a Homography (2D–2D) or a camera projection matrix (3D–2D, camera calibration) from a set of point correspondences.

Importance: DLT is the standard workhorse technique behind Homography estimation and camera calibration — it converts a projective (non-linear due to homogeneous scale) estimation problem into a linear least-squares problem solvable via SVD.

Applications: camera calibration, homography estimation for stitching/AR, pose estimation (PnP problem), general projective transformation estimation.

9.2 Core Concepts

9.2.1 Why 'Direct Linear'?

The relation x' ∼ Hx (or x ∼ PX) involves an unknown scale factor, which makes it non-linear in the usual sense. DLT eliminates the scale factor algebraically via the cross-product trick (x' × Hx = 0), producing equations that are linear in the unknown matrix entries — hence 'direct linear' transformation.

9.2.2 Homogeneous Linear Systems & SVD

Because DLT produces a homogeneous system Ah = 0 (right-hand side is zero — otherwise h = 0 would trivially 'solve' it), we cannot use ordinary least squares. Instead, we find the vector h (with ‖h‖ = 1) that minimizes ‖Ah‖ — this is exactly the right singular vector of A corresponding to its smallest singular value, obtained via SVD: A = $U\Sigma V^T$, and h = last column of V.

9.3 Mathematical Formulation (Homography case)

For each correspondence (x,y) ↔ (x',y'), using x' × Hx = 0, we get two independent linear equations:

$$\begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & x'x & x'y & x' \\ 0 & 0 & 0 & -x & -y & -1 & y'x & y'y & y' \end{bmatrix} h = 0$$

where h = (h11,h12,h13,h21,h22,h23,h31,h32,h33)^T. Stacking n ≥ 4 correspondences gives a 2n×9 matrix A; solve Ah = 0 via SVD.

9.3.1 DLT for Camera Calibration (3D–2D case)

Similarly, for world point X = (X,Y,Z,1) and image point x = (u,v,1) related by x ∼ PX (P is 3×4, 11 DoF), each correspondence gives a 2×12 pair of equations. Minimum 6 point correspondences needed (each gives 2 equations, 11 DoF → 5.5 → round up to 6).

9.4 Matrix Representation & Worked Numerical Example

Estimate a homography from 4 correspondences (simplified, axis-aligned case): (0,0)→(0,0), (1,0)→(2,0), (0,1)→(0,2), (1,1)→(2,2).

By inspection this is a pure scaling: x' = 2x, y' = 2y, so:

H =

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Verification using the DLT equation for point (1,0)→(2,0): row = (−1,0,−1,0,0,0,2,0,2). Check: −1(2)+0+(−1)(0)+0+0+0+2(0)+0(0)+2(1) = −2+0+2 = 0 ✓ — confirming the system is consistent with H above. Real data uses least-squares (SVD) because noise prevents exact satisfaction.

9.5 Algorithm: DLT

Input: ≥ 4 correspondences (homography) or ≥ 6 (camera matrix). Output: H (or P).

```
1. Normalize coordinates (Hartley normalization: centroid at origin, avg distance sqrt(2))
2. For each correspondence, build 2 rows of matrix A
3. Stack all rows -> A (2n x 9 for H, 2n x 12 for P)
4. Compute SVD: A = U * Sigma * V^T
5. h = last column of V (smallest singular value)
6. Reshape h into 3x3 (H) or 3x4 (P)
7. Denormalize the result
```

Time Complexity: O(n) to build A; SVD cost $O(9^2 \cdot n)$ (dominant for large n), or constant for a fixed small A dimension.

Python — general DLT for Homography and for the camera matrix P

```
import numpy as np

def dlt_homography(pts1, pts2):
    """DLT for a 2D-2D homography (same as estimate_homography in Topic 7)."""
    n = pts1.shape[0]
    A = np.zeros((2 * n, 9))
    for i in range(n):
        x, y = pts1[i]
        xp, yp = pts2[i]
        A[2*i]   = [-x, -y, -1, 0, 0, 0, xp*x, xp*y, xp]
        A[2*i+1] = [0, 0, 0, -x, -y, -1, yp*x, yp*y, yp]
    _, _, Vt = np.linalg.svd(A)
    H = Vt[-1].reshape(3, 3)
    return H / H[2, 2]

def dlt_camera_matrix(world_pts, image_pts):
    """
    DLT for the 3x4 camera projection matrix P (world_pts: Nx3, image_pts: Nx2, N>=6).
    """
    n = world_pts.shape[0]
    A = np.zeros((2 * n, 12))
    for i in range(n):
        X, Y, Z = world_pts[i]
        u, v = image_pts[i]
        A[2*i]   = [-X, -Y, -Z, -1, 0, 0, 0, 0, u*X, u*Y, u*Z, u]
        A[2*i+1] = [0, 0, 0, 0, -X, -Y, -Z, -1, v*X, v*Y, v*Z, v]
    _, _, Vt = np.linalg.svd(A)
    P = Vt[-1].reshape(3, 4)
    return P

# Worked example (pure scaling case)
pts1 = np.array([[0,0],[1,0],[0,1],[1,1]], dtype=float)
pts2 = np.array([[0,0],[2,0],[0,2],[2,2]], dtype=float)
H = dlt_homography(pts1, pts2)
print("Estimated H:\n", np.round(H, 3))   # -> approx [[2,0,0],[0,2,0],[0,0,1]]
```

9.6 Formula Sheet

- Homography DLT: 2n×9 system, min 4 points, solve via SVD.
- Camera matrix DLT: 2n×12 system, min 6 points, solve via SVD.

- Solution = smallest right-singular vector of A (last column of V in $A = U\Sigma V^T$).
- Always normalize coordinates before building A (Hartley normalization) for numerical stability.

9.7 Applications

Camera calibration (Topic 11), homography estimation for panorama stitching, planar pose estimation (AR), general PnP (Perspective-n-Point) problems, robotics hand-eye calibration.

9.8 Advantages & Limitations

Advantages: converts a non-linear projective estimation problem into a simple linear algebra problem; closed-form (no iterative optimization needed); fast. Limitations: does not minimize a geometrically meaningful error (minimizes algebraic error, not reprojection error) — often refined afterward via non-linear least squares (Levenberg-Marquardt / bundle adjustment); sensitive to noise and outliers without normalization/RANSAC.

9.9 Common Mistakes

- Skipping coordinate normalization — causes severe numerical ill-conditioning.
- Forgetting the minimum point requirement (4 for H, 6 for P).
- Taking the largest singular vector instead of the smallest (common exam slip).
- Confusing algebraic error minimization (what DLT does) with geometric/reprojection error minimization (what refinement does).

9.10 Key Points / Revision

- DLT = 'linearize the homogeneous projective equation, then solve via SVD.'
- Homography: 4 points minimum, 2n×9 system. Camera matrix: 6 points minimum, 2n×12 system.
- Always normalize input coordinates first; always take the smallest singular vector.
- DLT gives an initial estimate — typically refined with non-linear optimization for best accuracy.

10. RANSAC (Random Sample Consensus)

10.1 Introduction

Definition: RANSAC is an iterative, robust model-fitting algorithm designed to estimate model parameters (e.g., Homography, Fundamental matrix) accurately even when the data contains a large fraction of outliers (wrong correspondences).

Importance: real-world feature correspondences (from SIFT/ORB matching, etc.) always contain mismatches. Least-squares/DLT methods are extremely sensitive to even a few outliers; RANSAC provides robustness by fitting the model only to the 'consensus' of inlier data.

Applications: Homography estimation, Fundamental matrix estimation, line/plane fitting, object detection with noisy matches, SLAM front-ends.

10.2 Core Concepts

10.2.1 Inliers vs Outliers

- Inliers: data points consistent with the correct underlying model (within some error threshold).
- Outliers: data points that do not fit the model (wrong correspondences, noise, mismatches).

10.2.2 Core Idea

Instead of using all data points to fit a model at once (which is skewed by outliers), RANSAC repeatedly:

- Picks a small random subset of points (minimal set needed to fit the model, e.g., 4 for homography).
- Fits the model to just that subset.
- Counts how many remaining points agree with this model (within a threshold) — the consensus set / inlier count.
- Keeps the model with the largest consensus set found over many iterations.
- Optionally re-fits the model using all inliers of the best set (least-squares refinement).

10.3 Mathematical Formulation

10.3.1 Number of Iterations Required

Given: p = desired probability that RANSAC picks at least one all-inlier sample (confidence, e.g., 0.99); w = probability that a randomly selected point is an inlier (inlier ratio); n = minimum number of points needed to fit the model (e.g., 4 for homography, 8 for fundamental matrix).

The probability that a sample of n points is all inliers is w^n. After k iterations, the probability that all samples failed (had ≥1 outlier) is $(1-w^n)^k$. We want this to be small: $(1-w^n)^k = 1-p$, so:

$$k = log(1 - p) / log(1 - w^n)$$

Variables: k = number of iterations needed, p = desired confidence, w = inlier ratio, n = minimal sample size.

10.4 Solved Numerical Example

For homography estimation (n=4), inlier ratio w=0.6, desired confidence p=0.99. Find the required number of iterations k.

Step 1: $w^4 = 0.1296$. Step 2: $1-w^4 = 0.8704$. Step 3: $1-p = 0.01$.

$$k = log(0.01) / log(0.8704) = -4.605 / -0.1389 \approx 33.15$$

Answer: approximately 34 iterations are required (round up).

Sensitivity check: if the inlier ratio drops to w=0.3: w^4=0.0081, k = log(0.01)/log(0.9919) ≈ 566 iterations — showing RANSAC needs far more iterations as outlier contamination increases.

10.5 Algorithm: RANSAC

Input: set of correspondences, model type (e.g., homography), distance threshold t, confidence p. Output: best model parameters, inlier set.

```
best_inliers = {}
k = compute_iterations(p, w_estimate, n)
for i = 1 to k:
    sample = randomly select n points
    model = fit_model(sample)
    inliers = {}
    for each point in dataset:
        if distance(point, model) < t:
            inliers.add(point)
    if size(inliers) > size(best_inliers):
        best_inliers = inliers
        best_model = model
# Final refinement
final_model = fit_model(best_inliers)   # least squares on all inliers
return final_model, best_inliers
```

Time Complexity: O(k·N) where k = iterations, N = total number of points; k itself depends on the inlier ratio w as derived above.

Python — RANSAC iteration count and a generic RANSAC loop for Homography

```
import numpy as np
import math
import random

def ransac_iterations(p, w, n):
    """k = log(1-p) / log(1-w**n)"""
    return math.log(1 - p) / math.log(1 - w**n)

def ransac_homography(pts1, pts2, threshold=3.0, p=0.99, w_estimate=0.5, n=4):
    """
    Robustly estimate a homography from noisy correspondences (pts1 -> pts2).
    Uses dlt_homography() and apply_homography() from Topic 7/9.
    """
    num_points = pts1.shape[0]
    k = int(math.ceil(ransac_iterations(p, w_estimate, n)))
    best_inliers, best_H = [], None

    for _ in range(k):
        idx = random.sample(range(num_points), n)
        H_candidate = dlt_homography(pts1[idx], pts2[idx])

        inliers = []
        for i in range(num_points):
            x_proj = apply_homography(H_candidate, pts1[i])
            error = np.linalg.norm(np.array(x_proj) - pts2[i])
            if error < threshold:
                inliers.append(i)

        if len(inliers) > len(best_inliers):
            best_inliers, best_H = inliers, H_candidate

    # Final refinement: refit on all inliers
```

```
    final_H = dlt_homography(pts1[best_inliers], pts2[best_inliers])
    return final_H, best_inliers

# Worked example: iteration count from the notes
k = ransac_iterations(p=0.99, w=0.6, n=4)
print(f"Iterations needed: {k:.2f} -> {math.ceil(k)}")   # -> ~33.15 -> 34
```

10.6 Formula Sheet

- Iterations: k = log(1−p) / log(1−w^n)
- w = inlier ratio, n = minimal sample size, p = desired confidence
- Larger n or smaller w → many more iterations needed (exponential sensitivity)

10.7 Applications

Homography/Fundamental matrix estimation for stitching & SfM, robust line/plane fitting, object pose estimation with noisy keypoints, LIDAR plane segmentation in autonomous vehicles, robotics SLAM outlier rejection.

10.8 Advantages & Limitations

Aspect	Details
Advantages	Very robust to outliers, works even with >50% outliers, simple to implement, model-agnostic
Limitations	Non-deterministic (random sampling), computationally expensive for low inlier ratios, needs a good distance threshold t (hard to tune), performance degrades sharply with very high outlier ratios

10.9 Common Mistakes

- Choosing the minimal sample size n incorrectly (must equal the exact minimum points needed, e.g., 4 for H, 8 for F).
- Forgetting the final refinement step (re-fitting on all inliers, not just the minimal sample).
- Choosing threshold t too tight (rejects valid inliers) or too loose (accepts outliers).
- Confusing w (inlier ratio) with p (confidence) — a very common exam mix-up.

10.10 Key Points / Revision

- RANSAC = 'sample minimal set → fit → count inliers → repeat → keep best → refine.'
- Iteration formula k = log(1−p)/log(1−w^n) is a favorite university numerical question.
- Higher outlier ratio (lower w) or larger minimal set n dramatically increases required iterations.
- Always finish with a least-squares refit on the full inlier set for the final answer.

11. Camera Calibration

11.1 Introduction

Definition: Camera calibration is the process of estimating a camera's intrinsic parameters (focal length, principal point, distortion) and extrinsic parameters (rotation, translation relative to a world frame) from known correspondences (usually a calibration pattern like a checkerboard).

Importance: all metric 3D vision tasks (depth estimation, 3D reconstruction, AR overlay) require an accurately calibrated camera model; without calibration, pixel measurements cannot be converted to real-world geometry.

Applications: robotics (hand-eye calibration), autonomous vehicles (multi-camera rigs), AR/VR (accurate overlay registration), photogrammetry, industrial machine vision/inspection.

11.2 Core Concepts

11.2.1 Intrinsic Parameters

Internal camera properties independent of pose — focal length (fx, fy), principal point (cx, cy), skew (s) — captured in matrix K (see Topic 1).

11.2.2 Extrinsic Parameters

Camera's pose relative to the world — rotation R and translation t (see Topic 1).

11.2.3 Lens Distortion Models

Real lenses deviate from the ideal pinhole model, introducing geometric distortion.

Radial distortion (most significant, from lens curvature):

$$x_distorted = x(1 + k1\ r^2 + k2\ r^4 + k3\ r^6), \quad y_distorted = y(1 + k1\ r^2 + k2\ r^4 + k3\ r^6)$$

where $r^2 = x^2 + y^2$ (normalized image coordinates), k1, k2, k3 = radial distortion coefficients.

- k1 < 0: barrel distortion (image bulges outward)
- k1 > 0: pincushion distortion (image pinches inward)

Tangential distortion (from lens/sensor misalignment):

$$x_distorted = x + [2p1\ xy + p2(r^2 + 2x^2)]$$

$$y_distorted = y + [p1(r^2 + 2y^2) + 2p2\ xy]$$

11.2.4 Zhang's Calibration Method (most commonly examined)

A practical, widely-used technique using a planar checkerboard pattern observed from multiple orientations (typically ≥3 images):

- Detect checkerboard corners in each image (known 3D positions on the Z=0 plane, unknown pose per image).
- For each image, estimate a homography H between the planar pattern and the image (via DLT, Topic 9).
- Use constraints from multiple homographies (orthonormality of rotation columns) to solve for the intrinsic matrix K in closed form.
- Recover extrinsics [R|t] for each image from K and each H.
- Refine all parameters (including distortion) via non-linear least-squares (minimizing reprojection error).

11.3 Mathematical Formulation

Full camera model (from Topic 1): s·x = K[R|t]X, P = K[R|t]

Zhang's homography-intrinsics relation: for each calibration image, since the pattern is planar (Z=0), the projection simplifies to a homography:

$$H = K\,[r1 \;\; r2 \;\; t]$$

where r1, r2 are the first two columns of R. Using the orthonormality constraints r1 ⊥ r2 and |r1| = |r2|, one derives two equations per image in terms of $K^{-T}K^{-1}$ (the absolute conic image), and stacking ≥3 images gives enough equations to solve for all 5 intrinsic parameters.

11.4 Matrix Representation

P(3×4) = K(3×3)[R|t](3×4). Degrees of freedom: 5 intrinsic (or up to 4 if skew assumed 0 and fx=fy) + 6 extrinsic (3 rotation + 3 translation) per view = up to 11 total DoF, matching P's 11 DoF (Topic 1).

11.5 Solved Numerical Example

A checkerboard corner at world coordinates Xw = (0.05, 0.03, 0) m (planar pattern, Z=0) is imaged with R = I, t = $(0,0,0.5)^T$ m and:

K =

$$\begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

Find the pixel location.

Step 1 — Camera coordinates: Xc = R·Xw + t = (0.05, 0.03, 0.5)

Step 2 — Apply K: K·Xc = (800(0.05)+320(0.5), 800(0.03)+240(0.5), 0.5) = (200, 144, 0.5)

Step 3 — Normalize: u = 200/0.5 = 400, v = 144/0.5 = 288

Answer: the corner projects to pixel (400, 288).

11.6 Algorithm: Zhang's Calibration

Input: ≥3 images of a planar checkerboard pattern from different orientations. Output: K, distortion coefficients, [R|t] per image.

```
1. Detect known checkerboard corner correspondences in each image
2. Estimate homography H_i for each image via DLT (Topic 9)
3. Build constraint equations from R orthonormality -> solve linear system for K^-T K^-1
4. Recover K via Cholesky decomposition
5. Recover R_i, t_i per image from K^-1 * H_i
6. Estimate distortion coefficients (linear approx)
7. Refine ALL parameters jointly via non-linear least-squares (minimize reprojection error)
```

Time Complexity: dominated by the non-linear refinement (iterative, e.g., Levenberg–Marquardt), typically O(iterations × points).

Python — camera projection check and radial distortion model

```
import numpy as np

def project_point_full(K, R, t, Xw):
    """Same projection as Topic 1 (reused here for calibration checks)."""
```

```
    Xw = np.asarray(Xw, dtype=float).reshape(3, 1)
    t = np.asarray(t, dtype=float).reshape(3, 1)
    Xc = R @ Xw + t
    x_h = K @ Xc
    return x_h[0, 0] / x_h[2, 0], x_h[1, 0] / x_h[2, 0]

def apply_radial_distortion(x, y, k1, k2=0.0, k3=0.0):
    """Apply radial distortion to normalized image coordinates (x, y)."""
    r2 = x**2 + y**2
    factor = 1 + k1 * r2 + k2 * r2**2 + k3 * r2**3
    return x * factor, y * factor

def recover_extrinsics_from_homography(K, H):
    """
    Zhang's method: given K and a planar homography H = K[r1 r2 t],
    recover [r1 r2 t] and build the full rotation via r3 = r1 x r2.
    """
    K_inv = np.linalg.inv(K)
    h1, h2, h3 = H[:, 0], H[:, 1], H[:, 2]
    lam = 1.0 / np.linalg.norm(K_inv @ h1)
    r1 = lam * (K_inv @ h1)
    r2 = lam * (K_inv @ h2)
    r3 = np.cross(r1, r2)
    t = lam * (K_inv @ h3)
    R = np.column_stack([r1, r2, r3])
    return R, t

# Worked example from the notes
K = np.array([[800, 0, 320], [0, 800, 240], [0, 0, 1]], dtype=float)
u, v = project_point_full(K, np.eye(3), [0, 0, 0.5], [0.05, 0.03, 0])
print(f"Pixel: ({u:.0f}, {v:.0f})")   # -> (400, 288)
```

11.7 Formula Sheet

- $P = K[R|t]$
- Radial distortion: $x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$
- Tangential distortion: $x_d = x + 2p_1 xy + p_2(r^2 + 2x^2)$
- Zhang: $H = K[r_1\ r_2\ t]$ per planar view, ≥3 views needed
- $r_1 \perp r_2$, $|r_1| = |r_2|$ (orthonormality constraints)

11.8 Applications

Robotics (accurate manipulation via calibrated cameras), autonomous vehicles (multi-camera/LIDAR-camera fusion), AR/VR (precise virtual object registration), medical imaging (calibrated endoscopes), industrial inspection/metrology.

11.9 Advantages & Limitations

Advantages: Zhang's method is practical (simple checkerboard, no special equipment), gives both intrinsics and extrinsics, well-established/standard in OpenCV and similar toolkits. Limitations: requires good quality corner detection; assumes a rigid, accurately-known planar pattern; distortion model may not fully capture complex lens artifacts (e.g., fisheye needs a different model); sensitive to insufficient view diversity.

11.10 Common Mistakes

- Using too few or too similar (non-diverse angle) calibration images — leads to a poor/ambiguous K estimate.

- Forgetting to model radial distortion, causing systematic reprojection errors especially near image borders.
- Confusing intrinsic (camera-only, fixed) with extrinsic (pose, changes per image) parameters.
- Not refining the DLT/linear estimate with non-linear least-squares (reprojection error minimization).

11.11 Key Points / Revision

- P = K[R|t]; calibration recovers both K (intrinsic) and [R|t] (extrinsic, per view).
- Zhang's method: planar pattern, ≥3 views, homography per view, then closed-form + non-linear refinement.
- Radial distortion is the dominant lens error; barrel (k1<0) vs pincushion (k1>0).
- Frequently confused: intrinsic vs extrinsic parameters — intrinsic is fixed per camera, extrinsic varies with camera pose/image.

12. Histogram Processing

12.1 Introduction

Definition: A histogram represents the frequency distribution of pixel intensity values in an image. Histogram processing techniques (equalization, matching, stretching) manipulate this distribution to enhance contrast or match a desired appearance.

Applications: medical image contrast enhancement, satellite image processing, automatic exposure/contrast correction, image comparison/matching pipelines.

12.2 Core Concepts

12.2.1 Histogram

$$h(rk) = nk$$

where rk = the k-th intensity level, nk = number of pixels with that intensity.

12.2.2 PDF (Probability Density Function)

$$p(rk) = nk \,/\, (M{\cdot}N)$$

where M×N = total number of pixels in the image. p(rk) is an estimate of the probability of intensity level rk occurring.

12.2.3 CDF (Cumulative Distribution Function)

$$CDF(rk) = \Sigma\ (j{=}0\ to\ k)\ p(rj)$$

The running sum of the PDF up to level k — this is the key function used to compute histogram equalization.

12.3 Histogram Equalization

Goal: transform the image so its output histogram is approximately uniform, maximizing global contrast.

$$sk = (L{-}1) \times CDF(rk) = (L{-}1)\ \Sigma\ (j{=}0\ to\ k)\ p(rj)$$

Variables: sk = new (equalized) intensity, L−1 = maximum intensity value (255 for 8-bit), CDF(rk) = cumulative probability up to level rk.

Assumption: the transformation is monotonically increasing (preserves intensity ordering) and treats the histogram as a probability distribution to be flattened.

12.3.1 Histogram Matching (Specification)

Instead of flattening to uniform, histogram matching transforms an image so its histogram matches a specified target histogram (e.g., from a reference image).

Procedure:

- Compute CDF of the input image: CDF_in
- Compute CDF of the target/reference histogram: CDF_target
- For each input intensity, find the output intensity s such that CDF_target(s) = CDF_in(r) (invert the target CDF).

12.4 Histogram Equalization vs Histogram Matching

Property	Histogram Equalization	Histogram Matching
Target distribution	Uniform	Any specified reference histogram
Use case	General contrast enhancement	Matching appearance to a reference image
Steps	Single CDF computation	Two CDFs + CDF inversion
Control	No control over output shape	Full control via reference image
Example use	X-ray contrast boosting	Making satellite images from different dates look consistent

12.5 Solved Numerical Example — Histogram Equalization

A 3-bit image (L=8 levels, 0–7) has 64 total pixels with histogram nk = [8,10,6,12,8,10,6,4] for rk = 0..7.

Step 1 — PDF (p=nk/64): [0.125, 0.156, 0.094, 0.188, 0.125, 0.156, 0.094, 0.063]

Step 2 — CDF (cumulative sum): [0.125, 0.281, 0.375, 0.563, 0.688, 0.844, 0.938, 1.000]

Step 3 — New intensity sk = 7×CDF, rounded: [1, 2, 3, 4, 5, 6, 7, 7]

Result: input level 0 → output level 1, input level 1 → output level 2, and so on — the equalized image now has its pixel values spread more evenly across the full 0–7 range.

12.6 Formula Sheet

- Histogram: h(rk) = nk
- PDF: p(rk) = nk/MN
- CDF: CDF(rk) = Σ(j=0..k) p(rj)
- Equalization: sk = (L−1)·CDF(rk)
- Matching: find s such that CDF_target(s) = CDF_in(r)

12.7 Algorithm: Histogram Equalization

Input: grayscale image, L intensity levels. Output: contrast-enhanced image.

```
1. Compute histogram h(r_k) for all levels k = 0..L-1
2. Compute PDF: p(r_k) = h(r_k) / (M*N)
3. Compute CDF: cumulative sum of PDF
4. Compute mapping: s_k = round((L-1) * CDF(r_k))
5. Replace every pixel of intensity r_k with s_k
```

Time Complexity: O(M·N) (single pass to build the histogram + single pass to remap pixels).

Python — histogram equalization and histogram matching

```
import numpy as np

def histogram_equalization(image, L=256):
    """image: 2D array of integer intensities in [0, L-1]."""
    hist, _ = np.histogram(image, bins=L, range=(0, L))
    pdf = hist / image.size
    cdf = np.cumsum(pdf)
    mapping = np.round((L - 1) * cdf).astype(int)
    return mapping[image], mapping

def histogram_matching(image, reference, L=256):
    """Match 'image' histogram to 'reference' histogram."""
    hist_in, _ = np.histogram(image, bins=L, range=(0, L))
```

```
    hist_ref, _ = np.histogram(reference, bins=L, range=(0, L))
    cdf_in = np.cumsum(hist_in / image.size)
    cdf_ref = np.cumsum(hist_ref / reference.size)

    # For each input level, find the ref level whose CDF is closest
    mapping = np.zeros(L, dtype=int)
    for r in range(L):
        diffs = np.abs(cdf_ref - cdf_in[r])
        mapping[r] = np.argmin(diffs)
    return mapping[image], mapping

# Worked example from the notes
nk = np.array([8, 10, 6, 12, 8, 10, 6, 4])
pdf = nk / 64
cdf = np.cumsum(pdf)
sk = np.round(7 * cdf).astype(int)
print("PDF:", np.round(pdf, 3))
print("CDF:", np.round(cdf, 3))
print("Equalized levels sk:", sk)   # -> [1 2 3 4 5 6 7 7]
```

12.8 Applications

Medical imaging (X-ray/CT/MRI contrast enhancement), satellite/remote sensing (normalize across scenes), surveillance (enhance low-contrast night footage), preprocessing for feature detection/matching, AR/VR (consistent brightness across frames).

12.9 Advantages & Limitations

Aspect	Details
Advantages	Simple, automatic (no manual parameter tuning), effective global contrast enhancement
Limitations	Can over-enhance noise in flat regions, ignores spatial/local content, may produce unnatural-looking results, not ideal for non-uniform lighting (use CLAHE instead, Topic 3)

12.10 Common Mistakes

- Forgetting to round the mapped intensity sk to the nearest valid integer level.
- Confusing PDF (per-level probability) with CDF (cumulative probability) — CDF is used in the actual equalization formula, not PDF directly.
- Applying global histogram equalization when local (CLAHE) is more appropriate.
- Forgetting the (L−1) scaling factor in the equalization formula.

12.11 Key Points / Revision

- CDF is the workhorse function — memorize $s_k = (L-1)\text{CDF}(r_k)$.
- Histogram matching = equalize input AND target, then map via inverse of target CDF.
- Contrast stretching (Topic 3) is a simpler linear alternative; HE is non-linear and data-driven.
- Frequently confused: PDF vs CDF — PDF is the raw normalized histogram; CDF is its running sum, and it is CDF (not PDF) that directly gives the equalization mapping.